

在 Vertex Pipelines 執行自訂模型訓練作業

來源：<https://codelabs.developers.google.com/vertex-pipelines-custom-model?hl=zh-tw>

步驟數：9

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [設定雲端環境](#)
4. [設定 Vertex Pipelines](#)
5. [設定自訂模型訓練工作](#)
6. [設定批次預測工作](#)
7. [使用預先建構的元件建構管道](#)
8. [瞭解管道執行情況](#)
9. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何使用 Kubeflow Pipelines SDK，在 [Vertex Pipelines](#) 上執行自訂模型訓練工作。

課程內容

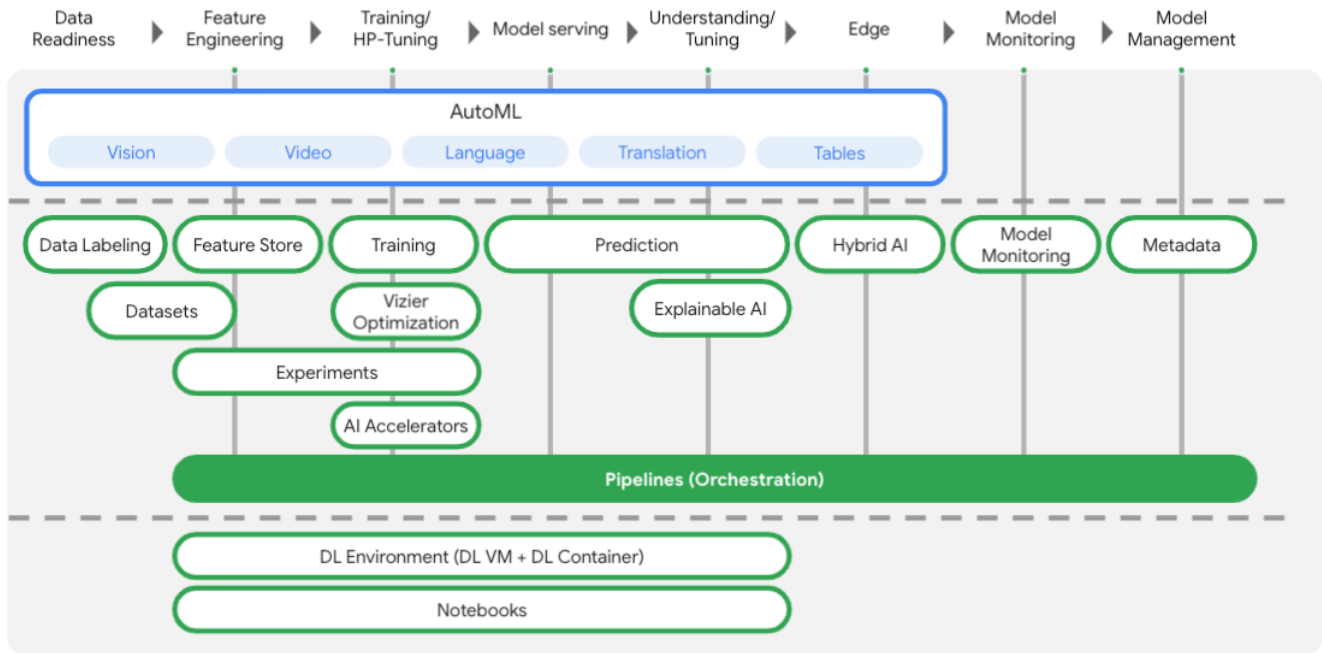
內容如下：

- 使用 Kubeflow Pipelines SDK 建構可擴充的機器學習 pipeline
- 建立及容器化自訂 Scikit-learn 模型訓練工作，該工作會使用 Vertex AI 代管資料集，並在 pipeline 內的 Vertex AI 訓練上執行
- 在 Vertex Pipelines 中執行批次預測工作
- 使用透過 `google_cloud_pipeline_components` 程式庫的預先建構元件，與 Vertex AI 服務互動

在 Google Cloud 上執行這個實驗室的總費用約為 **\$5 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 中的端對端代管式機器學習平台 [Vertex AI](#)。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。除了模型訓練和部署服務，Vertex AI 也提供各種 MLOps 產品，包括 Vertex Pipelines (本實驗室的重點)、模型監控、特徵儲存庫等。下圖列出所有 Vertex AI 產品。



如有任何意見，請參閱[支援頁面](#)。

機器學習 pipeline 有什麼用途？

深入探索前，我們先介紹使用 pipeline 的原因。假設您在建構機器學習工作流程，流程中會處理資料、訓練模型、調整超參數、評估及開發模型。每個步驟可能有不同的依附元件，如果光看整個工作流程，可能難以掌握這些步驟。開始擴展機器學習流程時，您或許會想與團隊成員分享機器學習工作流程，方便他們執行流程及貢獻程式碼。這時如果沒有可靠且可重複使用的程序，可能會難以進行。Pipeline 可將機器學習流程的各步驟化為容器，讓您開發個別步驟，並用可重複使用的方法，追蹤各步驟的輸入和輸出內容。您也可以根據 Cloud 環境的其他事件 (例如：有新的訓練資料時)，排定或觸發 pipeline 執行作業。

簡而言之：Pipeline 可協助您簡化及重複使用機器學習工作流程。

3. 設定雲端環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟動 Cloud Shell

在本實驗室中，您將在 [Cloud Shell](#) 工作階段中操作，這是由 Google Cloud 中執行的虛擬機器代管的指令解譯器。您也可以在本機電腦上輕鬆執行本節內容，但使用 Cloud Shell 可讓所有人在一致的環境中，獲得可重現的體驗。完成實驗室後，歡迎在自己的電腦上重試這個部分。

首次在 Cloud Shell 中發出需要授權的要求時，您會看到「授權 Cloud Shell」對話方塊 (如下所示)，請放心點選「授權」按鈕。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

Authorize

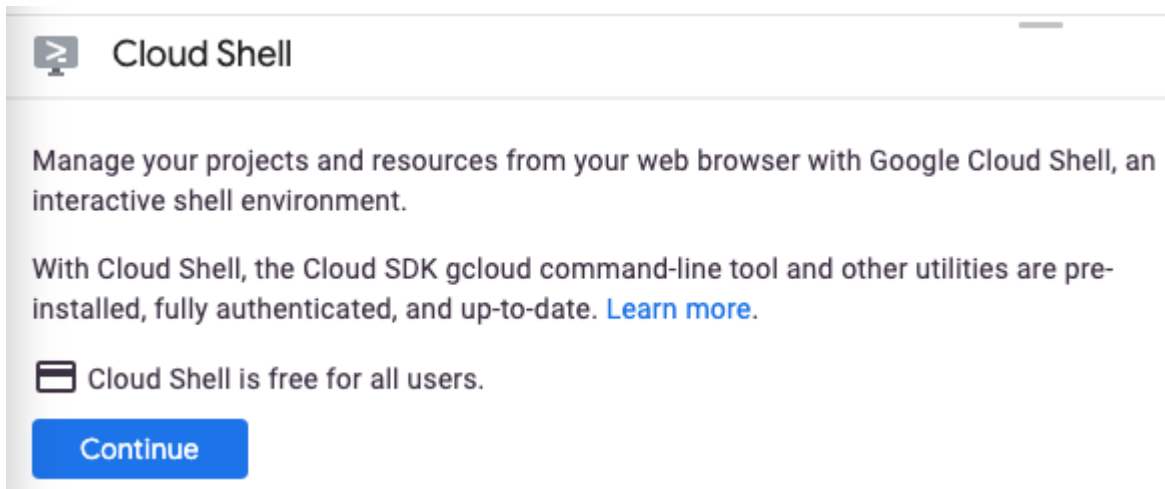
Reject

啟用 Cloud Shell

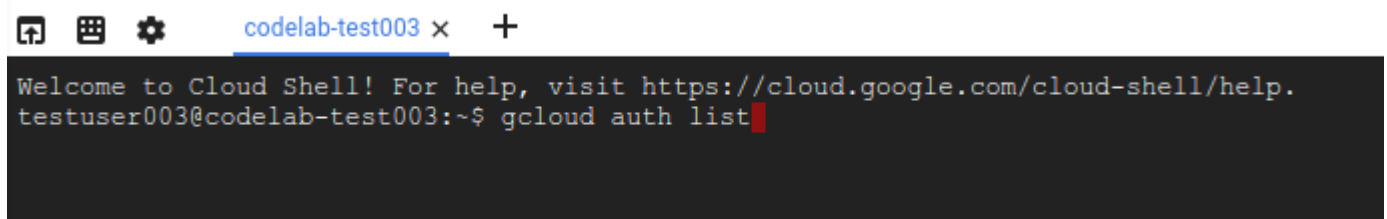
在 Cloud 控制台的右上角，按一下下方的按鈕來**啟用 Cloud Shell**：



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於需捲動位置)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



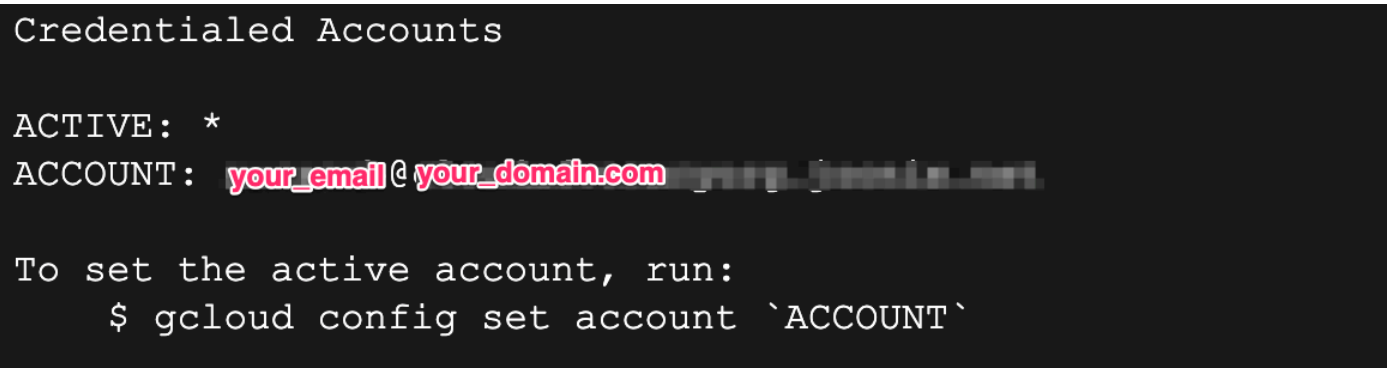
這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出內容應如下所示：



注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

Cloud Shell 有幾個環境變數，包括 `GOOGLE_CLOUD_PROJECT`，其中包含目前雲端專案的名稱。我們會在整個實驗室的各個地方使用這個檔案。您可以執行下列指令來查看：

```
echo $GOOGLE_CLOUD_PROJECT
```

步驟 2：啟用 API

在後續步驟中，您會瞭解需要這些服務的原因和位置，但現在請執行下列指令，授予專案 Compute Engine、Container Registry 和 Vertex AI 服務的存取權：

```
gcloud services enable compute.googleapis.com \  
                        containerregistry.googleapis.com \  
                        aiplatform.googleapis.com
```

這時應該會顯示類似以下的成功訊息：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished  
successfully.
```

步驟 3：建立 Cloud Storage 值區

如要在 Vertex AI 上執行訓練工作，我們需要儲存空間值區來儲存已儲存的模型資產。值區必須是區域值區。我們在這裡使用 `us-central`，但您也可以使用其他區域 (只要在本實驗室中替換即可)。如果已有 bucket，可以略過這個步驟。

在 Cloud Shell 終端機中執行下列指令，建立 bucket：

```
BUCKET_NAME=gs://$GOOGLE_CLOUD_PROJECT-bucket  
gsutil mb -l us-central1 $BUCKET_NAME
```

接著，我們會授予運算服務帳戶這個值區的存取權。這樣可確保 Vertex Pipelines 具備將檔案寫入這個值區的必要權限。執行下列指令來新增這項權限：

```
gcloud projects describe $GOOGLE_CLOUD_PROJECT > project-info.txt
PROJECT_NUM=$(cat project-info.txt | sed -nre 's:.*projectNumber\: (.*):\1:p')
SVC_ACCOUNT="${PROJECT_NUM//\'}-compute@developer.gserviceaccount.com"
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT --member
serviceAccount:$SVC_ACCOUNT --role roles/storage.objectAdmin
```

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



Vertex AI



Dashboard



Datasets



Features



Labelling tasks



Workbench



Pipelines



Training



Experiments



Models



Endpoints

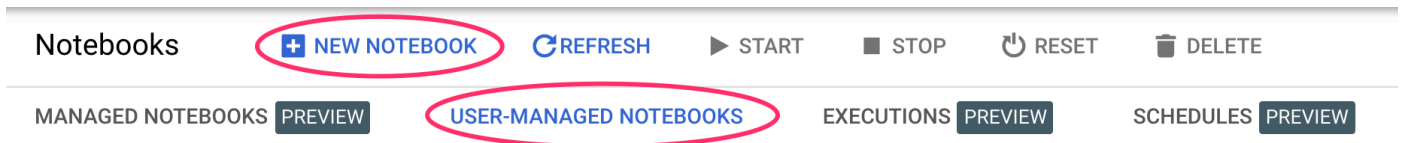


Batch predictions

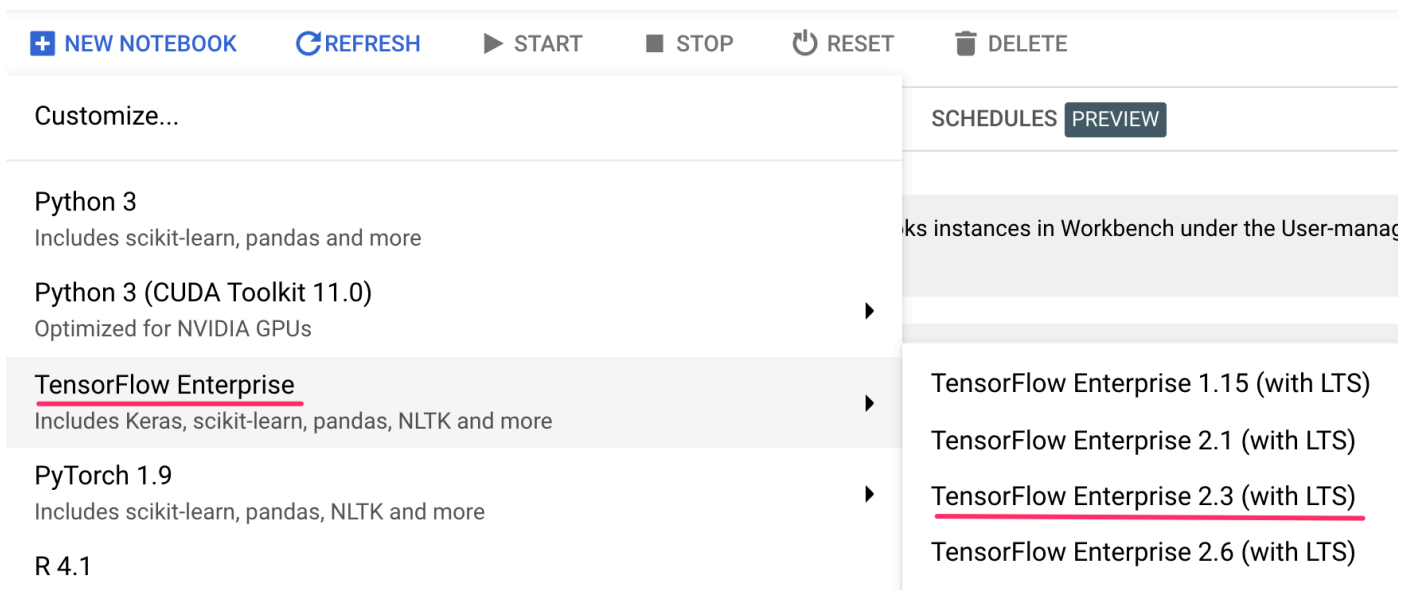


Metadata

然後在「使用者自行管理的 Notebooks」中，按一下「New Notebook」(新增 Notebook)：



然後選取「TensorFlow 企業版 2.3 (含 LTS)」執行個體類型，且不加入任何 GPU：



使用預設選項，然後按一下「建立」。

步驟 5：開啟筆記本

建立執行個體後，請選取「Open JupyterLab」：



tensorflow-2-3-
20210405-151653

OPEN JUPYTERLAB

步驟 4 · 約 5 分鐘

4. 設定 Vertex Pipelines

使用 Vertex Pipelines 前，需先安裝幾個額外的程式庫：

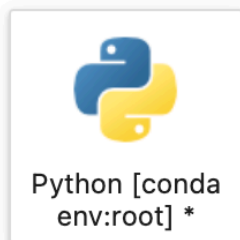
- **Kubeflow Pipelines**：這是用於建構 pipeline 的 SDK。Vertex Pipelines 可執行透過 Kubeflow Pipelines 和/或 TFX 建構的 pipeline。
- **Google Cloud Pipeline Components** 提供預先建立的元件，可讓您在建構 pipeline 時，輕鬆操作 Vertex AI 服務。

步驟 1：建立 Python 筆記本及安裝程式庫

首先，從筆記本執行個體的啟動器選單 (點選筆記本左上方的「+」圖示即可開啟)，選取「Python 3」建立筆記本：



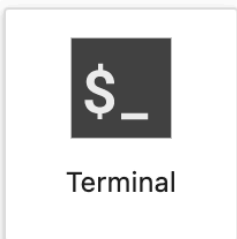
Notebook



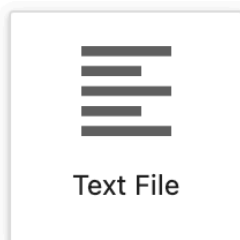
Console



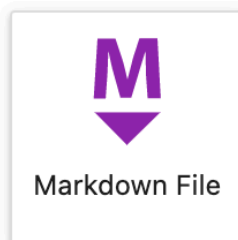
Other



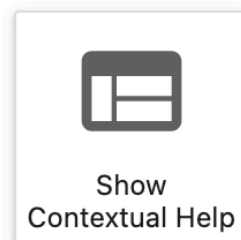
Terminal



Text File



Markdown File



Show
Contextual Help

點選筆記本執行個體左上方的「+」符號，即可打開啟動器選單。

如要安裝本實驗室使用的兩項服務，請先在筆記本儲存格設定使用者旗標：

```
USER_FLAG = "--user"
```

接著，從筆記本執行下列指令：

```
!pip3 install {USER_FLAG} google-cloud-aiplatform==1.7.0 --upgrade
!pip3 install {USER_FLAG} kfp==1.8.9 google-cloud-pipeline-components==0.2.0
```

安裝輸出內容中可能會顯示一些警告訊息。

安裝這些套件後，需重新啟動核心：

```
import os

if not os.getenv("IS_TESTING"):
    # Automatically restart kernel after installs
    import IPython

    app = IPython.Application.instance()
    app.kernel.do_shutdown(True)
```

最後，確認套件是否已正確安裝。KFP SDK 版本應為 1.8 以上：

```
!python3 -c "import kfp; print('KFP SDK version: {}'.format(kfp.__version__))"
!python3 -c "import google_cloud_pipeline_components; print('google_cloud_pipeline_components
version: {}'.format(google_cloud_pipeline_components.__version__))"
```

步驟 2：設定專案 ID 和 bucket

您會在本實驗室用到自己的雲端專案 ID，以及先前建立的 bucket。接下來，請為 ID 和 bucket 建立變數。

如果不知道自己的專案 ID，執行以下指令或許就能取得：

```
import os
PROJECT_ID = ""

# Get your Google Cloud project ID from gcloud
if not os.getenv("IS_TESTING"):
    shell_output=!gcloud config list --format 'value(core.project)' 2>/dev/null
    PROJECT_ID = shell_output[0]
    print("Project ID: ", PROJECT_ID)
```

否則，請在這裡設定：

```
if PROJECT_ID == "" or PROJECT_ID is None:
    PROJECT_ID = "your-project-id" # @param {type:"string"}
```

接著建立用來儲存 bucket 名稱的變數。如果您是在本實驗室中建立，下列做法就適用。否則，您必須手動設定：

```
BUCKET_NAME="gs://" + PROJECT_ID + "-bucket"
```

步驟 3：匯入程式庫

新增以下程式碼，匯入本程式碼研究室會用到的程式庫：

```
from kfp.v2 import compiler, dsl
from kfp.v2.dsl import pipeline

from google.cloud import aiplatform
from google_cloud_pipeline_components import aiplatform as gcc_aip
```

步驟 4：定義常數

建構 pipeline 前，最後還需定義幾項常數變數。PIPELINE_ROOT 是 Cloud Storage 路徑，pipeline 建立的構件會寫入到這個位置。此程式碼使用 us-central1 做為區域，但如果您在建立 bucket 時使用不同的區域，請更新下方程式碼中的 REGION 變數：

```
PATH=%env PATH
%env PATH={PATH}:/home/jupyter/.local/bin
REGION="us-central1"

PIPELINE_ROOT = f"{BUCKET_NAME}/pipeline_root/"
PIPELINE_ROOT
```

執行上方的程式碼後，您應該會看到 pipeline 的根目錄。這是要寫入 pipeline 構件的 Cloud Storage 位置。格式為 gs://YOUR-BUCKET-NAME/pipeline_root/

5. 設定自訂模型訓練工作

設定管道前，我們需要先編寫自訂模型訓練工作的程式碼。如要訓練模型，我們會使用 UCI Machine Learning [乾豆資料集](#) (來源：KOKLU, M. and OZKAN, I.A., (2020), "Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques." In Computers and Electronics in Agriculture, 174, 105507. [DOI](#)。

第一個管道步驟會使用包含這個豆類資料版本的 BigQuery 資料表，在 Vertex AI 中建立代管資料集。資料集會做為訓練工作的輸入內容。在訓練程式碼中，我們將可存取環境變數，以存取這個受管理資料集。

以下是自訂訓練工作的設定方式：

- 編寫 Scikit-learn `DecisionTreeClassifier` 模型，將資料中的豆類分類
- 將訓練程式碼封裝在 Docker 容器中，並推送到 Container Registry

接著，我們就能直接從管道啟動 Vertex AI 訓練工作。立即開始！

步驟 1：在 Docker 容器中定義訓練程式碼

在筆記本執行個體中開啟啟動器，然後選取「終端機」：

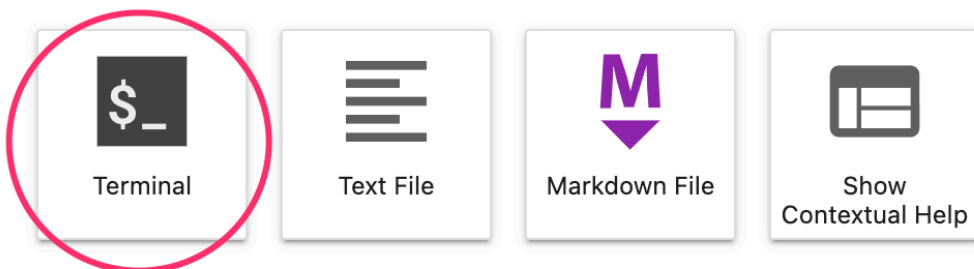
Notebook



Console



Other



接著執行下列指令，設定要新增容器化程式碼的目錄：

```
mkdir traincontainer
cd traincontainer
touch Dockerfile

mkdir trainer
touch trainer/train.py
```

執行這些指令後，左側應該會建立名為 `traincontainer/` 的目錄 (可能需要點選重新整理圖示才能看到)。您會在 `traincontainer/` 目錄中看到下列項目：

```
+ Dockerfile
+ trainer/
  + train.py
```

將程式碼容器化的第一步是建立 Dockerfile。在 Dockerfile 中，我們會納入執行映像檔所需的所有指令。這會安裝我們使用的所有程式庫，並設定訓練程式碼的進入點。開啟您剛建立的 Dockerfile，然後新增下列內容：

```
FROM gcr.io/deeplearning-platform-release/sklearn-cpu.0-23
WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

RUN pip install sklearn google-cloud-bigquery joblib pandas google-cloud-storage

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.train"]
```

如要在筆記本執行個體中編輯檔案時儲存檔案，可以使用 `ctrl+s`。

接著，開啟 `train.py` 檔案。我們將在這裡新增訓練程式碼。將下列內容複製到 `train.py`。這會從代管資料集擷取資料、將資料放入 Pandas DataFrame、訓練 Scikit-learn 模型，並將訓練好的模型上傳至 Cloud Storage：

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve
from sklearn.model_selection import train_test_split
from google.cloud import bigquery
from google.cloud import storage
from joblib import dump

import os
import pandas as pd

bqclient = bigquery.Client()
storage_client = storage.Client()

def download_table(bq_table_uri: str):
    prefix = "bq://"
    if bq_table_uri.startswith(prefix):
        bq_table_uri = bq_table_uri[len(prefix):]

    table = bigquery.TableReference.from_string(bq_table_uri)
    rows = bqclient.list_rows(
        table,
    )
    return rows.to_dataframe(create_bqstorage_client=False)

# These environment variables are from Vertex AI managed datasets
training_data_uri = os.environ["AIP_TRAINING_DATA_URI"]
test_data_uri = os.environ["AIP_TEST_DATA_URI"]

# Download data into Pandas DataFrames, split into train / test
df = download_table(training_data_uri)
test_df = download_table(test_data_uri)
labels = df.pop("Class").tolist()
data = df.values.tolist()
test_labels = test_df.pop("Class").tolist()
test_data = test_df.values.tolist()

# Define and train the Scikit model
skmodel = DecisionTreeClassifier()
skmodel.fit(data, labels)
score = skmodel.score(test_data, test_labels)
print('accuracy is:', score)

# Save the model to a local file
dump(skmodel, "model.joblib")

# Upload the saved model file to GCS
bucket = storage_client.get_bucket("YOUR_GCS_BUCKET")
model_directory = os.environ["AIP_MODEL_DIR"]
storage_path = os.path.join(model_directory, "model.joblib")
blob = storage.blob.Blob.from_string(storage_path, client=storage_client)
blob.upload_from_filename("model.joblib")

```

接著返回筆記本，執行下列指令，將上述指令碼中的 `YOUR_GCS_BUCKET` 替換為 Cloud Storage 值區名稱：

```
BUCKET = BUCKET_NAME[5:] # Trim the 'gs://' before adding to train script
!sed -i -r 's@YOUR_GCS_BUCKET@"$BUCKET"@' traincontainer/trainer/train.py
```

你也可以視需要手動執行這項操作。如果這麼做，請務必更新指令碼，並確保 bucket 名稱不含 `gs://`。

現在訓練程式碼已在 Docker 容器中，我們準備好在雲端執行訓練。

步驟 2：將容器推送至 Container Registry

訓練程式碼完成後，即可推送至 Google Container Registry。稍後設定 pipeline 的訓練元件時，我們會讓 Vertex Pipelines 指向這個容器。

返回終端機，在 `traincontainer/` 目錄的根層級中，使用 Container Registry 上的容器映像檔 URI 定義變數。

```
PROJECT_ID=$(gcloud config get-value project)
IMAGE_URI="gcr.io/$PROJECT_ID/scikit:v1"
```

接著，執行下列指令來建構容器：

```
docker build ./ -t $IMAGE_URI
```

由於訓練指令碼會使用 Vertex AI 環境變數，因此無法透過 `docker run` 在筆記本執行個體中在本機執行容器。請放心，這項功能可在雲端運作。

最後，將容器推送到 Container Registry：

```
docker push $IMAGE_URI
```

前往 Cloud 控制台的「Container Registry」專區，確認容器是否已上傳。如下所示：

scikit

gcr.io > Your Project ID > scikit

Filter Enter property name or value

☐	Name	Tags	Virtual size ?	Created
☐	 9cf2b8824192	v1	1.5 GB	Just now

步驟 6 · 約 2 分鐘

6. 設定批次預測工作

管道的最後一個步驟會執行批次預測工作。如要使用這項功能，我們需要在 Cloud Storage 中提供 CSV 檔案，內含要取得預測結果的範例。我們會在筆記本中建立這個 CSV 檔案，並使用 `gsutil` 指令列工具將檔案複製到 Cloud Storage。

將批次預測範例複製到 Cloud Storage

下列檔案包含豆類資料集中每個類別的 3 個範例。由於模型會預測 `class` 欄，因此下方範例未納入該欄。執行下列指令，在本機筆記本中建立這個 CSV 檔案：

%%writefile batch_examples.csv

Area,Perimeter,MajorAxisLength,MinorAxisLength,AspectRation,Eccentricity,ConvexArea,EquivDiameter,Extent,Solidity,roundness,Compactness,ShapeFactor1,ShapeFactor2,ShapeFactor3,ShapeFactor4

23288,558.113,207.567738,143.085693,1.450653336,0.7244336162,23545,172.1952453,0.8045881703,0.9890847314,0.9395021523,0.8295857874,0.008913077034,0.002604069884,0.6882125787,0.9983578734
23689,575.638,205.9678003,146.7475015,1.403552348,0.7016945718,24018,173.6714472,0.7652721693,0.9863019402,0.8983750474,0.8431970773,0.00869465998,0.002711119968,0.7109813112,0.9978994889

23727,559.503,189.7993849,159.3717704,1.190922235,0.5430731512,24021,173.8106863,0.8037601626,0.9877607094,0.952462433,0.9157600082,0.007999299741,0.003470231343,0.8386163926,0.9987269085

31158,641.105,212.0669751,187.1929601,1.132879009,0.4699241567,31474,199.1773023,0.7813134733,0.989959967,0.9526231013,0.9392188582,0.0068061806,0.003267009878,0.8821320637,0.9993488983
32514,649.012,221.4454899,187.1344232,1.183349841,0.5346736437,32843,203.4652564,0.7849831,0.9899826447,0.9700068737,0.9188051492,0.00681077351,0.002994124691,0.8442029022,0.9989873701
33078,659.456,235.5600775,178.9312328,1.316483846,0.6503915309,33333,205.2223615,0.7877214708,0.9923499235,0.9558229607,0.8712102818,0.007121351881,0.002530662194,0.7590073551,0.9992209221

33680,683.09,256.203255,167.9334938,1.525623324,0.7552213942,34019,207.081404,0.80680321,0.9900349805,0.9070392732,0.8082699962,0.007606985006,0.002002710402,0.6533003868,0.9966903078
33954,716.75,277.3684803,156.3563259,1.773951126,0.825970469,34420,207.9220419,0.7994819873,0.9864613597,0.8305492781,0.7496238998,0.008168948587,0.001591181142,0.5619359911,0.996846984
36322,719.437,272.0582306,170.8914975,1.591993952,0.7780978465,36717,215.0502424,0.7718560075,0.9892420405,0.8818487005,0.7904566678,0.007490177594,0.001803782407,0.6248217437,0.9947124371

36675,742.917,285.8908964,166.8819538,1.713132487,0.8119506999,37613,216.0927123,0.7788277766,0.9750618137,0.8350248381,0.7558572692,0.0077952528,0.001569528272,0.5713202115,0.9787472145
37454,772.679,297.6274753,162.1493177,1.835514817,0.8385619338,38113,218.3756257,0.8016695205,0.9827093118,0.7883332637,0.7337213257,0.007946480356,0.001420623993,0.5383469838,0.9881438654

37789,766.378,313.5680678,154.3409867,2.031657789,0.8704771226,38251,219.3500608,0.7805870567,0.9879218844,0.8085170916,0.6995293312,0.008297866252,0.001225659709,0.4893412853,0.9941740339

47883,873.536,327.9986493,186.5201272,1.758516115,0.822571799,48753,246.9140116,0.7584464543,0.9821549443,0.7885506623,0.7527897207,0.006850002074,0.00135695419,0.5666923636,0.9965376533
49777,861.277,300.7570338,211.6168613,1.42123379,0.7105823885,50590,251.7499649,0.8019106536,0.9839296304,0.843243269,0.8370542883,0.00604208839,0.001829706116,0.7006598815,0.9958014989
49882,891.505,357.1890036,179.8346914,1.986207449,0.8640114945,51042,252.0153467,0.7260210171,0.9772736178,0.7886896753,0.7055518063,0.007160679276,0.001094585314,0.4978033513,0.9887407248

53249,919.923,325.3866286,208.9174205,1.557489212,0.7666552108,54195,260.3818974,0.6966846347,0.9825445152,0.7907120655,0.8002231025,0.00611066177,0.001545654241,0.6403570138,0.9973491406

61129,964.969,369.3481688,210.9473449,1.750902193,0.8208567513,61796,278.9836198,0.7501135067,0.9892064211,0.8249553283,0.7553404711,0.006042110436,0.001213219664,0.5705392272,0.9989583843

61918,960.372,353.1381442,224.0962377,1.575832543,0.7728529173,62627,280.7782864,0.7539207091,0.9886790043,0.8436218213,0.7950947556,0.005703319619,0.00140599258,0.6321756704,0.9962029945

141953,1402.05,524.2311633,346.3974998,1.513380332,0.7505863011,143704,425.1354762,0.7147107987,0.9878152313,0.9074598849,0.8109694843,0.003692991084,0.0009853172185,0.6576715044,0.99530

71199

145285,1440.991,524.9567463,353.0769977,1.486805285,0.7400216694,146709,430.0960442,0.7860466
375,0.9902937107,0.8792413513,0.8192980608,0.003613289371,0.001004269363,0.6712493125,0.99801
70255
146153,1476.383,526.1933264,356.528288,1.475881001,0.7354662103,149267,431.3789276,0.73193609
78,0.9791380546,0.8425962592,0.8198107159,0.003600290972,0.001003163512,0.6720896099,0.991924
286

然後將檔案複製到 Cloud Storage bucket :

```
!gsutil cp batch_examples.csv $BUCKET_NAME
```

我們會在下一個步驟定義管道時，參考這個檔案。

7. 使用預先建構的元件建構管道

訓練程式碼已在雲端，現在可以從管道呼叫。我們定義的 pipeline 會使用先前安裝的

`google_cloud_pipeline_components` 程式庫中的三個預先建立元件。這些預先定義的元件可簡化設定管道所需的程式碼，並讓我們使用 Vertex AI 服務，例如模型訓練和批次預測。

如果找不到可完成所需工作的預先建構元件，可以定義自己的 Python 型自訂元件。如需範例，請參閱[這個程式碼研究室](#)。

三步驟管道會執行下列動作：

- 在 Vertex AI 中建立代管資料集
- 使用我們設定的自訂容器，在 Vertex AI 上執行訓練工作
- 對已訓練的 Scikit-learn 分類模型執行批次預測工作

步驟 1：定義管道

由於我們使用預先建立的元件，因此可以在管道定義中設定整個管道。在筆記本儲存格中新增下列程式碼：

```

@pipeline(name="automl-beans-custom",
          pipeline_root=PIPELINE_ROOT)
def pipeline(
    bq_source: str = "bq://sara-vertex-demos.beans_demo.large_dataset",
    bucket: str = BUCKET_NAME,
    project: str = PROJECT_ID,
    gcp_region: str = REGION,
    bq_dest: str = "",
    container_uri: str = "",
    batch_destination: str = ""
):
    dataset_create_op = gcc_aip.TabularDatasetCreateOp(
        display_name="tabular-beans-dataset",
        bq_source=bq_source,
        project=project,
        location=gcp_region
    )

    training_op = gcc_aip.CustomContainerTrainingJobRunOp(
        display_name="pipeline-beans-custom-train",
        container_uri=container_uri,
        project=project,
        location=gcp_region,
        dataset=dataset_create_op.outputs["dataset"],
        staging_bucket=bucket,
        training_fraction_split=0.8,
        validation_fraction_split=0.1,
        test_fraction_split=0.1,
        bigquery_destination=bq_dest,
        model_serving_container_image_uri="us-docker.pkg.dev/vertex-ai/prediction/sklearn-
cpu.0-24:latest",
        model_display_name="scikit-beans-model-pipeline",
        machine_type="n1-standard-4",
    )

    batch_predict_op = gcc_aip.ModelBatchPredictOp(
        project=project,
        location=gcp_region,
        job_display_name="beans-batch-predict",
        model=training_op.outputs["model"],
        gcs_source_uris=["{0}/batch_examples.csv".format(BUCKET_NAME)],
        instances_format="csv",
        gcs_destination_output_uri_prefix=batch_destination,
        machine_type="n1-standard-4"
    )

```

步驟 2：編譯及執行管道

定義 pipeline 後，即可編譯 pipeline。以下程式碼會產生用於執行 pipeline 的 JSON 檔案：

```

compiler.Compiler().compile(
    pipeline_func=pipeline, package_path="custom_train_pipeline.json"
)

```

接著，請建立 `TIMESTAMP` 變數。我們會在工作 ID 中使用這項資訊：

```

from datetime import datetime

TIMESTAMP = datetime.now().strftime("%Y%m%d%H%M%S")

```

接著定義管道工作，並傳遞幾個專案專屬的參數：

```

pipeline_job = aiplatform.PipelineJob(
    display_name="custom-train-pipeline",
    template_path="custom_train_pipeline.json",
    job_id="custom-train-pipeline-{}".format(TIMESTAMP),
    parameter_values={
        "project": PROJECT_ID,
        "bucket": BUCKET_NAME,
        "bq_dest": "bq://{0}".format(PROJECT_ID),
        "container_uri": "gcr.io/{0}/scikit:v1".format(PROJECT_ID),
        "batch_destination": "{0}/batchpredresults".format(BUCKET_NAME)
    },
    enable_caching=True,
)

```

最後，執行工作來建立新的管道執行作業：

```

pipeline_job.submit()

```

執行這個儲存格後，您應該會看到記錄，其中包含在控制台中查看管道執行的連結：

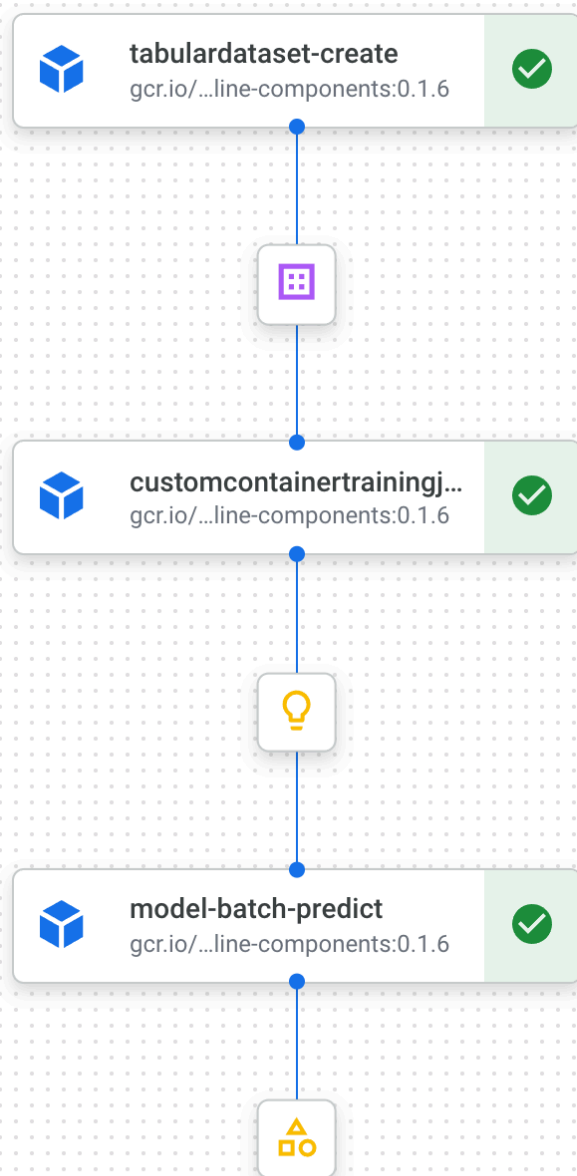
```

job.run()
INFO:google.cloud.aiplatform.pipeline_jobs:Creating PipelineJob
INFO:google.cloud.aiplatform.pipeline_jobs:PipelineJob created. Resource name: projects/[REDACTED]/locations/us-central1/pipelineJobs/hello-world-pipeline-20210920155317
INFO:google.cloud.aiplatform.pipeline_jobs:To use this PipelineJob in another session:
INFO:google.cloud.aiplatform.pipeline_jobs:pipeline_job = aiplatform.PipelineJob.get('projects/[REDACTED]/locations/us-central1/pipelineJobs/hello-world-pipeline-20210920155317')
INFO:google.cloud.aiplatform.pipeline_jobs:View Pipeline Job:
https://console.cloud.google.com/vertex-ai/locations/us-central1/pipelines/runs/hello-world-pipeline-20210920155317?project=[REDACTED]
INFO:google.cloud.aiplatform.pipeline_jobs:PipelineJob projects/[REDACTED]/locations/us-central1/pipelineJobs/hello-world-pipeline-20210920155317 current state:
PipelineState.PIPELINE_STATE_RUNNING
INFO:google.cloud.aiplatform.pipeline_jobs:PipelineJob projects/[REDACTED]/locations/us-central1/pipelineJobs/hello-world-pipeline-20210920155317 current state:
PipelineState.PIPELINE_STATE_RUNNING

```

Go to this URL

前往該連結。您也可以開啟[管道資訊主頁](#)來存取。完成後，您的管道應如下所示：



這個 pipeline 的執行時間為 **5 至 10 分鐘**，但您可以在執行完畢前繼續下一個步驟。接下來，您將進一步瞭解這些管道步驟中發生的情況。

8. 瞭解管道執行情況

接下來，我們將深入探討管道中的三個步驟。

管道步驟 1：建立代管資料集

管道的第一步是在 Vertex AI 中建立代管資料集。如果您在主控台的「管道」部分點選下列資料集連結：

← custom-train-pipeline-20210929175851

CLONE

⋮

Runtime Graph

2/3 steps completed

Expand Artifacts

90%

🔍 🔍 🔍

📄

1. Click on the dataset artifact

tabulardataset-create

gcr.io/...line-components:0.1.6

dataset

Type: system.Dataset

2. Click to view the dataset

VIEW LINEAGE

Name

dataset

Type

system.Dataset

URI

[aiplatform://v1/projects/462141068491/locations/us-central1/datasets/6885326531086778368](#)

您會在 Vertex AI 中看到資料集，其中包含 BigQuery 中資料來源的連結，以及資料集中不同資料欄的相關資訊。將代管資料集上傳至 Vertex AI 後，即可用於訓練 AutoML 或自訂模型。

對於使用代管資料集的自訂模型工作，Vertex AI 會將特殊環境變數傳遞至訓練工作，並處理將資料分割為訓練和測試集。我們會在下一個管道步驟中用到這個檔案。

管道步驟 2：在 Vertex AI 訓練中訓練模型

自訂訓練工作執行期間，您可以點選直接在 Vertex Pipelines 控制台中查看記錄：

custom-train-pipeline-20210929182539 [CLONE](#) [STOP](#) [DELETE](#)

Runtime Graph 1/3 steps completed Expand Artifacts 100%

1. Click on the custom training component

2. Click to view logs

Logs

MAIN JOB

Logs Showing 55 log entries Severity: Default Filter Filter logs

No older entries found matching current filter.

2021-09-29 14:26:30.333 EDT	Waiting for job to be provisioned.
2021-09-29 14:26:30.607 EDT	Waiting for training program to start.
2021-09-29 14:26:31.045 EDT	Job is preparing.
2021-09-29 14:26:31.343 EDT	Opening GCS connection...
2021-09-29 14:26:31.343 EDT	Mounting file system "gcsfuse"...
2021-09-29 14:26:31.345 EDT	File system has been successfully mounted.
2021-09-29 14:26:42.578 EDT	--init.display_name
2021-09-29 14:26:42.578 EDT	pipeline-beans-custom-train

您也可以在 Vertex AI [訓練資訊主頁](#) 中查看自訂訓練工作的詳細資料。訓練工作完成後，Vertex AI 會建立模型資源。接著，我們可以將這個模型部署至端點，以便進行線上預測，或是建立批次預測工作 (我們會在下一個管道步驟中執行這項操作)。

管道步驟 3：對模型執行批次預測工作

最後，我們的管道會根據透過 CSV 檔案傳入的範例進行預測。批次預測工作完成後，Vertex AI 會將 CSV 檔案寫入我們在 Cloud Storage 中指定的位置。這個管道步驟開始執行後，您可以前往 Vertex AI 控制台的「批次預測」專區，查看建立的工作。

工作完成後，按一下工作即可查看批次預測的 Cloud Storage 網址：

ID	7630129110672474112
Model	scikit-beans-model-pipeline
Objective	Custom
Import location	gs://sara-vertex-demos-bucket/batch_examples.csv
Total items	21
Predicted items	21
Created	Sep 29, 2021 at 01:59pm
Updated	Sep 29, 2021 at 02:17pm
Elapsed time	15 min 42 sec
Status	Completed without errors
Export location	gs://sara-vertex-demos-bucket/batchpredresults/prediction-scikit-beans-model-pipeline-2021_09_29T10_59_10_224Z

按一下該連結前往 Cloud Storage 目錄，即可找到預測結果，然後按一下下載其中一個 `prediction.results` 檔案。檔案中應會顯示類似下列的資料列：

```
{"instance": [33954.0, 716.75, 277.3684803, 156.3563259, 1.773951126, 0.825970469, 34420.0, 207.9220419, 0.7994819873, 0.9864613597, 0.8305492781, 0.7496238998, 0.008168948587, 0.001591181142, 0.5619359911, 0.996846984], "prediction": "HOROZ"}
```

包括特定執行個體的特徵值，以及模型預測的類別。在本例中，模型認為這是「HOROZ」豆。

這樣就完成了實驗室！

🎉 恭喜！ 🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 使用 Kubeflow Pipelines SDK 建構含有自訂元件的端對端 pipeline
- 在 Vertex Pipelines 上執行管道，並使用 SDK 啟動管道執行作業
- 在控制台中查看及分析 Vertex Pipelines 圖表
- 使用預先建立的 pipeline 元件，將 Vertex AI 服務新增至 pipeline
- 排定週期性管道工作

如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

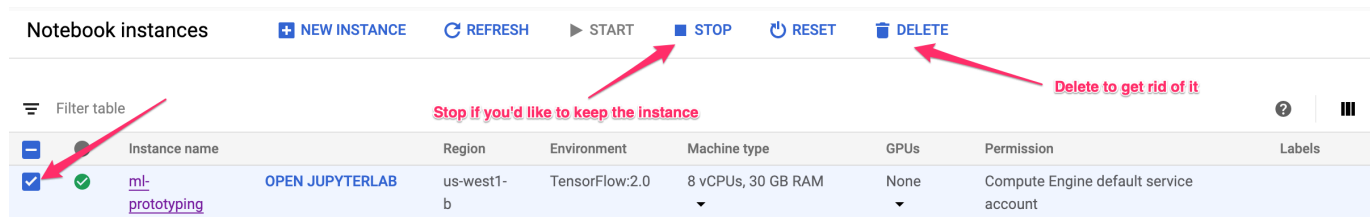
步驟 9 · 約 2 分鐘

9. 清除所用資源

為避免產生費用，建議您刪除在本實驗室中建立的資源。

步驟 1：停止或刪除 Notebooks 執行個體

如要繼續使用在本實驗室中建立的筆記本，建議您在未使用時關閉筆記本。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」。如要完全刪除執行個體，請選取「刪除」：



步驟 2：刪除 Cloud Storage bucket

如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

